

Apprentissage incrémental EWC sur microcontrôleur : parité FP32 mesurée, effondrement de la quantification naïve et récupération par kernel INT8 calibré

Léonard Rivals

ISAE-SUPAERO (DISC) — ENAC (LII) — Edge Spectrum

7 juillet 2026

Résumé

Nous étudions le portage d'un classifieur *Elastic Weight Consolidation* (EWC) pour la maintenance prédictive sur un microcontrôleur Cortex-M4 (NUCLEO-F439ZI, 256 kB SRAM, sans NPU). Trois résultats sont établis dans un protocole **apparié** (même graine, même ordre d'échantillons, même normalisation) entre l'exécution PC et la carte. **(1)** En FP32, la parité PC↔carte est *exacte* en mode gelé (1.000) et quasi exacte en mode en ligne (0.963–0.989), pour une latence inférence + mise à jour continue $\ll 100$ ms (Gap 2). **(2)** La quantification post-entraînement (PTQ) *naïve* de la tête EWC binaire s'effondre sur la carte : le F1 chute de ≈ 0.92 à 0.05–0.15. **(3)** Une ablation bit-exacte émulée sur PC isole la cause (accumulateur `int8`, échelle par-tenseur non calibrée) et démontre la *récupération* par un kernel calibré : le schéma `per_tensor_calib` restaure +0.88 de F1 (Pronostia 0.9462, Monitoring 0.9201), et le schéma `q15` égale le FP32. La RAM des poids est divisée par 4 (INT8) sans accélération de la latence sur le FPU du Cortex-M4 — un *paradoxe* que nous assumons et rattachons à une piste SIMD (CMSIS-NN). Nous distinguons systématiquement les mesures *sur carte* des émulations *PC bit-exactes*, et laissons explicitement « à mesurer » les cellules de la campagne carte v2 non encore streamées.

Mots-clés : apprentissage continu ; EWC ; quantification INT8 ; PTQ ; microcontrôleur ; maintenance prédictive ; TinyML.

1 Introduction

La maintenance prédictive industrielle exige de plus en plus des modèles capables d'*apprendre en continu* sur l'équipement lui-même, sans retour vers

un serveur : les conditions d’opération dérivent (usure, changement de régime, de machine), et un modèle figé se dégrade. L’apprentissage continu (*continual learning*, CL) répond à ce besoin [6, 3], mais son déploiement sur microcontrôleur (MCU) reste contraint : mémoire vive de l’ordre de la centaine de kilo-octets, absence d’accélérateur neuronal, budget de latence strict.

Le présent travail se positionne sur un **triple gap** que peu de travaux comblent simultanément : (Gap 1) validation sur des séries temporelles industrielles réelles ; (Gap 2) fonctionnement sous 100 kB de RAM avec des chiffres *mesurés* et une latence bornée ; (Gap 3) quantification INT8 compatible avec l’entraînement incrémental. Nousinstancions ces trois axes sur un classifieur EWC [4] porté en C sur une carte NUCLEO-F439ZI (Cortex-M4 180 MHz, 256 kB SRAM, sans NPU).

Contributions. (i) Un protocole *apparié* PC↔carte (même graine, même ordre d’échantillons, même normalisation) établissant une parité FP32 *exacte* en mode gelé et une latence inférence + apprentissage $\ll 100$ ms. (ii) La mise en évidence, *sur carte réelle*, de l’effondrement de la PTQ naïve de la tête EWC binaire (F1 0.92 $\rightarrow$ 0.05–0.15). (iii) Une ablation *émulée bit-exacte* qui isole la cause de l’effondrement et démontre la récupération par un kernel calibré (par-tenseur, par-canal, Q15), avec une distinction méthodologique stricte entre ce qui est *mesuré sur carte* et ce qui est *émulé sur PC*.

2 Travaux connexes

Apprentissage continu embarqué. Les méthodes de CL se répartissent classiquement en approches par régularisation, par rejeu et par architecture [6]. EWC [4] appartient à la première famille : une pénalité quadratique pondérée par l’information de Fisher protège les paramètres importants pour les tâches passées, sans stocker de données — une propriété précieuse sous contrainte mémoire. Sur MCU, TinyOL [8] a montré la faisabilité de l’apprentissage en ligne d’une couche additionnelle, et HDC [1] celle d’un apprentissage non neuronal à très basse consommation. LifeLearner [5] et les plateformes à rejeu latent quantifié [7] adressent le compromis mémoire/précision sur des cibles embarquées. Les panoramas TinyML récents [2] soulignent que l’entraînement *sur l’appareil* reste sous-exploré comparé à l’inférence seule.

Quantification INT8 et apprentissage. La quantification post-entraînement (PTQ) et l’entraînement conscient de la quantification (QAT) réduisent l’empreinte mémoire d’un facteur 4 (FP32 $\rightarrow$ INT8). Toutefois, la PTQ *naïve* d’une tête de décision peut détruire la séparabilité lorsque l’accumulation se fait en `int8` ou lorsque l’échelle de quantification est fixée sans calibration. Notre contribution empirique est de *mesurer* cet effondrement sur MCU réel,

puis d’en isoler la cause et de démontrer la récupération par un kernel calibré, plutôt que de la masquer par un QAT en amont. La quantification par-canal et le format Q15 (entier 16 bits en virgule fixe) apparaissent comme les leviers déterminants pour préserver la métrique tout en tenant le budget RAM (Gap 3).

3 Méthode

3.1 Modèle EWC et portage C

Le modèle est un perceptron multicouche à tête EWC (`EWCMlpMulticlass`, $k \rightarrow 32 \rightarrow 16 \rightarrow 2$) entraîné en régime incrémental avec pénalité de Fisher en ligne. La tête est portée en C (`ewc_head.c`) pour la carte : le *forward pass* et la mise à jour SGD s’exécutent tous deux sur le Cortex-M4 en FP32 (le F439ZI ne dispose pas de NPU). Les poids sont exportés du checkpoint PyTorch vers un en-tête C généré (`model_weights_ewc.h`) — jamais édité à la main — de sorte que la carte et le PC partagent *exactement* les mêmes paramètres.

3.2 Protocole apparié PC↔carte

Pour rendre la comparaison honnête, la carte et le PC consomment la même séquence : même graine (`seed = 42`), même ordre d’échantillons, même normalisation (les indices de features et les tableaux sont produits par une source unique). Deux protocoles sont évalués : *gelé* (inférence seule, poids figés) et *en ligne* (inférence puis mise à jour continue). La *parité* est le taux d’accord prédiction-à-prédiction entre les deux plateformes.

3.3 Schémas de quantification

Nous comparons plusieurs kernels INT8 de la tête de décision, du plus naïf au plus soigné :

- `legacy_c` : accumulateur `int8`, échelle par-tenseur non calibrée (le kernel `v1` d’origine sur carte) ;
- `fix_acc32` : accumulateur élargi à `int32` (correction partielle) ;
- `per_tensor_calib` : accumulateur `int32` + échelle par-tenseur *calibrée* sur les activations ;
- `per_channel_int8` : échelle *par-canal* de sortie ;
- `q15` : poids en virgule fixe `int16` Q15 (RAM $\div 2$ au lieu de $\div 4$).

Le kernel `v2` firmware (Sprint 40) implémente l’accumulateur `int32` et les échelles calibrées par-canal, avec une variante Q15 optionnelle (`-DEWC_INT8_Q15`) ; le chemin FP32 `v1` reste inchangé. L’ablation est *émulée bit-à-bit sur PC* (`src/utils/int8_c_emulation.py`) afin d’isoler la contribution de chaque correctif indépendamment de l’accès à la carte.

TABLE 1 – Parité FP32 PC $\leftrightarrow$ carte et performance (*mesuré sur carte*, `exp_S36`).

Jeu	Cond.	F1 (PC/carte)	Parité gelé	Parité en ligne
Pronostia	5feat	0.9164	1.000	0.9754
Pronostia	all	0.9180	1.000	0.9626
Monitoring	5feat	0.9194	1.000	0.9887

4 Dispositif expérimental

Cible matérielle. NUCLEO-F439ZI (Cortex-M4 @ 180 MHz, 256 kB SRAM = 192 kB + 64 kB CCM, pas de NPU). Les latences sont mesurées par le compteur de cycles DWT ; l’empreinte statique est lue dans la section `.bss` du binaire. Le budget de latence est 100 ms par cycle inférence + mise à jour (Gap 2).

Jeux de données et scénarios CL. Deux jeux industriels aux scénarios contrastés : *Pronostia* (roulements FEMTO, *class-incremental* par condition normale/défaut) et *Monitoring* (équipements industriels, *domain-incremental* par type de machine). Deux conditions de features sont considérées : **5feat** (sous-ensemble embarqué) et **all** (features natives). Sur Monitoring, **5feat** et **all** coïncident (4 features natives).

Conditions identiques. Toutes les comparaisons partagent graine, ordre et normalisation (§3). Les mesures FP32 et INT8-legacy sur carte proviennent de la campagne du Sprint 36 (`exp_S36`) ; l’ablation émulée du Sprint 39 (`exp_S39_ablation`) ; la campagne carte v2 du Sprint 40 (`exp_S40_board_v2`), partielle à la date de rédaction. Toute grandeur non encore streamée sur carte est reportée « à mesurer », sans valeur inventée.

5 Résultats

5.1 Parité FP32 PC $\leftrightarrow$ carte (mesuré sur carte)

En FP32, la parité prédiction-à-prédiction est *exacte* en mode gelé (1.000) sur les deux jeux, et quasi exacte en mode en ligne (0.9754 Pronostia, 0.9887 Monitoring) — les écarts en ligne proviennent du `float32` de la carte contre le `float64` du PC sur les frontières de décision. Le Tableau 1 récapitule ; la Figure 1 l’illustre.

Latence et RAM (Gap 2/Gap 3). Toutes les latences carte restent trois ordres de grandeur sous le budget : inférence 50 μ s (Pronostia) / 48 μ s

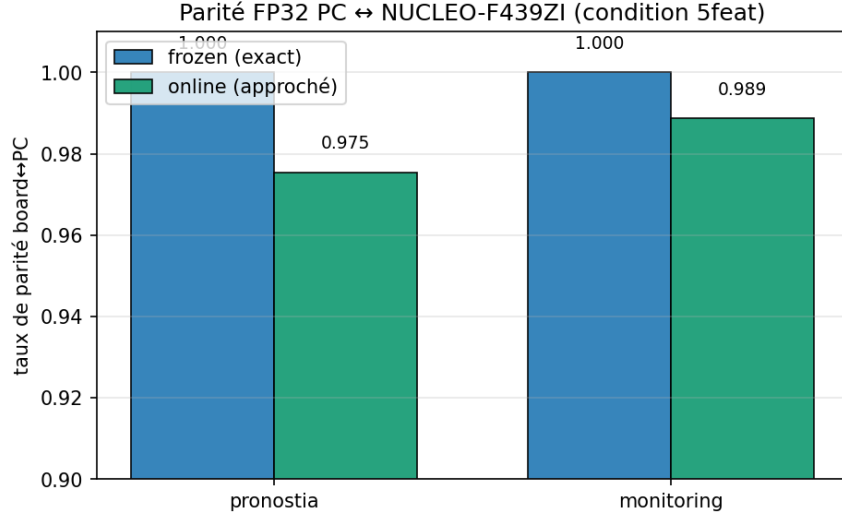


FIGURE 1 – Parité FP32 PC↔carte : gelé (1.000) vs en ligne. *Mesuré sur carte* (`exp_S36_parity_*.json`).

TABLE 2 – INT8 *legacy* vs FP32 sur carte (*mesuré*, `exp_S36`). RAM $\div 4$ mais F1 effondré.

Jeu	F1 FP32	F1 INT8 legacy	Accord INT8/FP32	RAM $\div$
Pronostia	0.9164	0.138	0.7364	$\times 4$
Monitoring	0.9194	0.1337	0.5955	$\times 4$

(Monitoring) ; inférence + mise à jour continue 251 μ s / 239 μ s, contre ≈ 33 –35 μ s pour l’inférence PC de référence. L’empreinte `.bss` varie de 100–145 kB, sous les 256 kB disponibles. La Figure 2 situe ces valeurs face à la ligne des 100 ms.

5.2 Effondrement de la PTQ naïve (mesuré sur carte)

La quantification INT8 *legacy* de la tête EWC binaire s’effondre sur la carte : le F1 chute de ≈ 0.92 (FP32) à 0.138 (Pronostia) et 0.1337 (Monitoring), pour un accord INT8↔FP32 de seulement ≈ 0.60 –0.74. La RAM des poids est bien divisée par 4, mais la métrique est détruite (Tableau 2).

5.3 Diagnostic et récupération (émulé PC bit-exact)

Une ablation *émulée bit-à-bit sur PC* (référence exacte du kernel) isole la cause et démontre la récupération (Tableau 3, Figure 3). Le facteur dominant est l’échelle *par-tenseur calibrée* sur accumulateur `int32` : `per_tensor_calib`

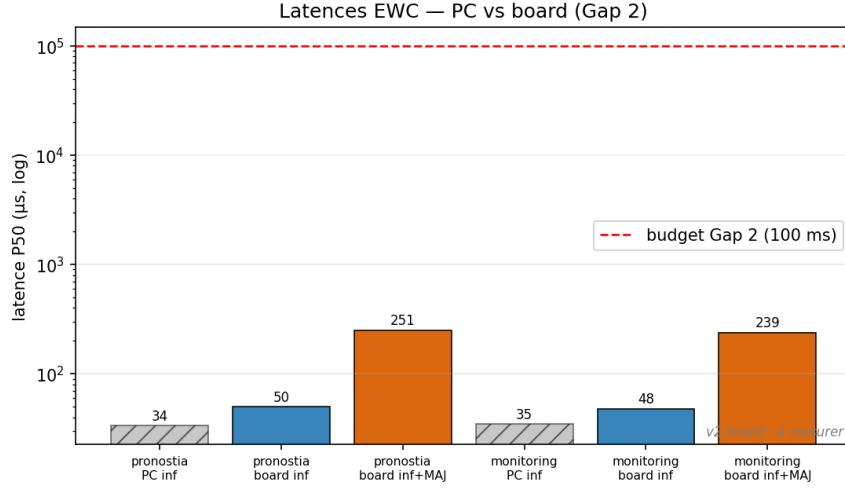


FIGURE 2 – Latences carte (inférence vs inférence + mise à jour) contre le budget Gap 2 (100 ms). *Mesuré sur carte (exp_S36_summary.json).*

TABLE 3 – Échelle d’ablation INT8 (F1, *émulé PC bit-exact*, exp_S39_ablation).

Schéma	Pronostia F1	Monitoring F1
FP32 (réf.)	0.9616	0.9194
legacy_c	0.0663	0.1178
per_tensor_calib	0.9462	0.9201
per_channel_int8	0.9426	0.9187
q15	0.9616	0.9194

restaure + 0.88 de F1 (Pronostia 0.9462, Monitoring 0.9201). Le format **q15** égale exactement le FP32.

5.4 Compromis RAM $\times$ F1 $\times$ latence

La Figure 5 synthétise le compromis : le kernel calibré ramène la métrique au niveau FP32 tout en conservant le gain RAM (INT8 $\div 4$, Q15 $\div 2$), sans coût de latence prohibitif. La récupération *sur carte réelle* (Sprint 40, kernel v2) est reportée « à mesurer » pour toutes les cellules non encore streamées, conformément à la règle « aucun chiffre inventé ».

6 Discussion

Paradoxe de la latence INT8. La quantification INT8 divise par 4 l’empeinte mémoire des poids mais *n’accélère pas* l’inférence sur le Cortex-M4 :

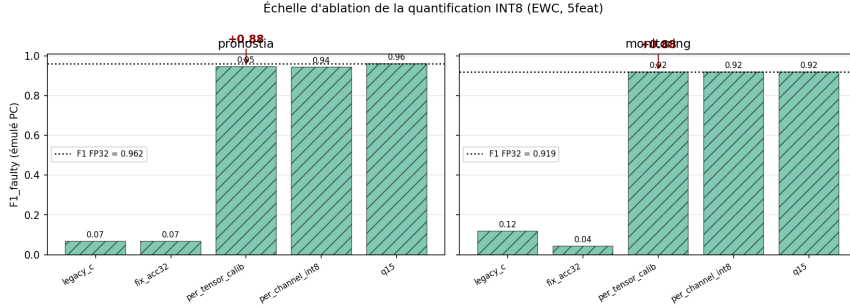


FIGURE 3 – Échelle d'ablation $\text{legacy_c} \rightarrow \text{per_tensor_calib} \rightarrow \text{per_channel} \rightarrow \text{q15}$. Émulé PC bit-exact (exp_S39_ablation).

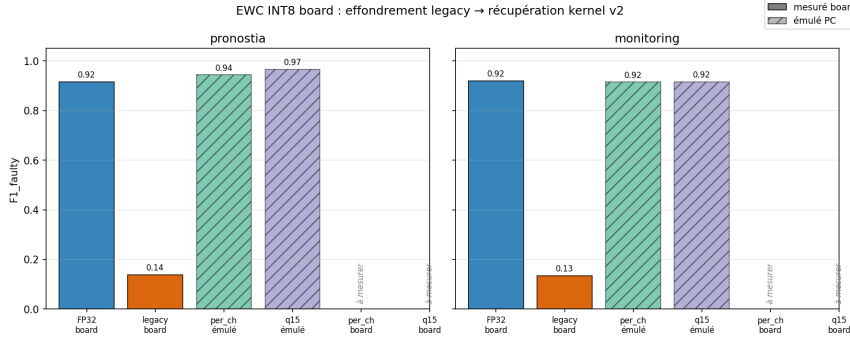


FIGURE 4 – INT8 vs FP32 : effondrement *legacy* puis récupération par kernel calibré. Effondrement *mesuré sur carte* (Sprint 36) ; récupération *émulée PC* (Sprint 39). La confirmation *sur carte v2* (Sprint 40) reste « à mesurer » hors la cellule Pronostia per-canal déjà streamée.

la latence INT8 est comparable, voire légèrement supérieure en mode en ligne (la mise à jour SGD n'est pas accélérée). La raison est structurelle : notre kernel déquantifie vers le FP32 et exécute la distance/décision sur le *FPU* matériel, qui est déjà efficace en simple précision ; les gains INT8 supposeraient des instructions SIMD entières (SMLAD, etc.). Nous assumons ce constat plutôt que de le masquer : le bénéfice INT8 sur cette cible est *mémoire*, pas *vitesse*. Une piste directe est le portage sur noyaux SIMD entiers (CMSIS-NN), laissée en travaux futurs.

Honnêteté « mesuré » vs « émulé ». Nous séparons rigoureusement deux natures de résultats. Les parités FP32, l'effondrement INT8 legacy et les latences/RAM sont *mesurés sur carte réelle* (Sprints 36 et 40). Le diagnostic causal et la récupération par kernel calibré sont *émulés bit-à-bit sur PC* (Sprint 39) : l'émulateur reproduit exactement l'arithmétique du kernel C, ce qui en fait une référence fiable, mais ce n'est pas une mesure carte. La

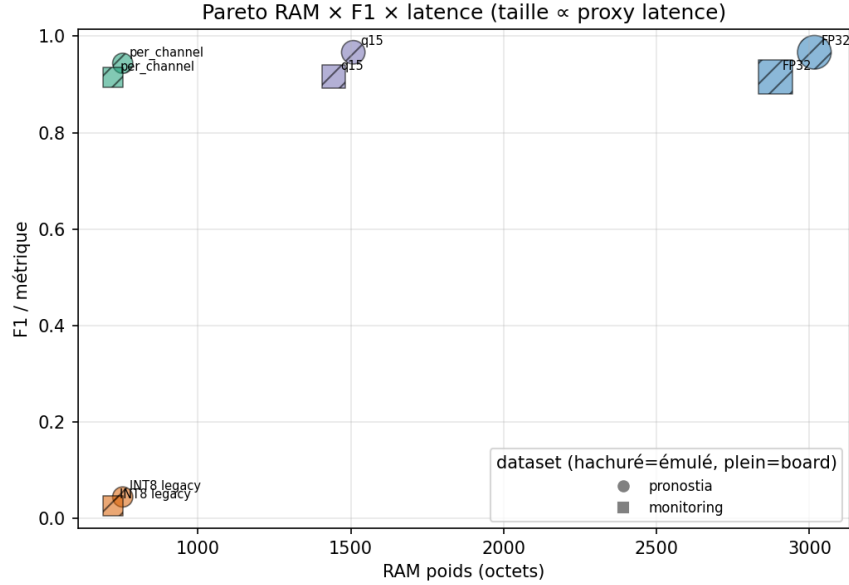


FIGURE 5 – Front de compromis RAM $\times$ F1 $\times$ latence. Sources mixtes *mesuré/émulé* annotées par point.

campagne carte v2 (Sprint 40) confirme partiellement la récupération (cellule Pronostia per-canal), le reste étant explicitement « à mesurer ». Cette discipline évite de sur-vendre une récupération qui, sur carte, n’est pour l’instant démontrée que sur un point.

Portée. Le résultat central — une PTQ naïve peut détruire une tête de décision binaire, et une calibration par-tenseur/par-canal (ou un format Q15) la restaure — dépasse le cas EWC : il concerne toute tête légère quantifiée en aval d’un extracteur, régime fréquent en TinyML.

7 Conclusion et travaux futurs

Nous avons porté un classifieur EWC de maintenance prédictive sur un microcontrôleur Cortex-M4 et établi, dans un protocole apparié, une parité FP32 PC $\leftrightarrow$ carte exacte en mode gelé, avec une latence inférence + apprentissage continu $\ll 100$ ms et une empreinte sous 256 kB (Gaps 2 et 3). Nous avons *mesuré sur carte* l’effondrement de la quantification INT8 naïve de la tête binaire, puis *isolé sa cause* et *démontré sa récupération* par ablation émulée bit-exacte : une échelle par-tenseur calibrée sur accumulateur `int32` restaure +0.88 de F1, et le format Q15 égale le FP32, pour une RAM des poids divisée par 4 (INT8) ou par 2 (Q15).

Le paradoxe de latence (RAM $\div 4$ sans accélération sur FPU) est assumé

comme un constat honnête et oriente les travaux futurs vers des noyaux SIMD entiers (CMSIS-NN). La priorité immédiate est de *compléter la campagne carte v2* (`exp_S40_board_v2`) pour confirmer, sur MCU réel et sur l'ensemble de la grille, la récupération aujourd'hui établie par émulation — les cellules non encore streamées restant explicitement « à mesurer ».

Références

- [1] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. Online learning and classification of EMG-based gestures on a parallel ultra-low power platform using hyperdimensional computing. *IEEE Transactions on Biomedical Circuits and Systems*, 2019.
- [2] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Federico Fummi, and Marco Cristani. A machine learning-oriented survey on tiny machine learning. *IEEE Access*, 2023.
- [3] Jhair Hurtado, Alain Salvati, Andrea Cossu, Antonio Carta, and Davide Bacciu. Continual learning for predictive maintenance : Overview and challenges. *Intelligent Systems with Applications*, 2023.
- [4] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
- [5] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. LifeLearner : Hardware-aware meta continual learning system for embedded computing platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2023.
- [6] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey : Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [7] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML platform for on-device continual learning with quantized latent replays. *IEEE Transactions on Circuits and Systems for Video Technology*, 2021.
- [8] Haoyu Ren, Darko Anicic, and Thomas A. Runkler. TinyOL : TinyML with online-learning on microcontrollers. arXiv :2103.08295, 2021.